
Market_Place

API Testing Report

Swagger Documentation and Endpoint Testing

Course

Internet Programming CSE341

Prepared by:

Jana Hany Elshafie 22P0235

Moaz Mohamed Zakaria 22P0307

Mohamed yehia zakaria 22p0064

Seif elhusseiny 22P0215

Mohamed Ahmed Abdelhamid 22P0287

Kirollos Ramzy 22P0194

Presented to:

Dr. Islam Tharwat

Eng.Hamdy Osama

1. Introduction

Market_Place is a full-stack e-commerce marketplace built using the MERN stack (MongoDB, Express.js, React, Node.js). The platform supports two user roles — buyers and sellers — enabling a complete purchase lifecycle from product discovery, cart management, and checkout through to order fulfillment and post-purchase reviews.

The backend API provides RESTful endpoints for authentication, product management, cart and wishlist operations, order placement and tracking, user reviews, and a reporting system. All endpoints are secured using JSON Web Tokens (JWT) and enforce role-based access control.

API testing is a critical step in software development. It validates that each endpoint behaves correctly, returns the expected status codes and data structures, enforces authentication and authorization rules, and handles errors gracefully. Systematic testing also serves as living documentation for the system and helps detect regressions when the codebase evolves.

This report documents the testing process carried out using Swagger UI — an interactive API documentation and testing tool integrated directly into the backend. Every endpoint was executed with appropriate request bodies and authentication tokens, and the results were captured as screenshots.

2. Tools and Technologies

The following technologies and tools were used throughout the development and testing of the Market_Place API:

Tool / Technology	Description
Node.js	JavaScript runtime environment used to run the backend server.
Express.js	Minimal and flexible Node.js web application framework used to define API routes and middleware.
MongoDB	NoSQL document database used to store all application data including users, products, orders, reviews, and flags.
Mongoose	Object Data Modeling (ODM) library for MongoDB, providing schema validation and query abstraction.
React / Vite	Frontend framework and build tool used for the buyer and seller web applications.
JWT (JSON Web Token)	Stateless authentication mechanism. Tokens are issued on login and sent with each protected API request in the Authorization header.
Swagger UI	Interactive API documentation and testing tool, integrated at http://localhost:4000/api-docs , generated using swagger-jsdoc and swagger-ui-express.
Browser / Postman	Used as alternative clients for manual testing and verification where needed.

3. Backend Setup

The backend server is a Node.js application powered by Express.js, connecting to a MongoDB Atlas cloud database. Before any API testing can be performed, the backend must be running locally.

3.1 Starting the Backend Server

Open a terminal, navigate to the backend folder, and run the following command:

```
cd Market_Place/backend
node src/server.js
```

Upon successful startup, the terminal will display the following confirmation message:

```
API running on http://localhost:4000
```

The backend server listens on port 4000. It connects to the MongoDB Atlas cluster specified in the .env file using the MONGODB_URI environment variable. If the connection fails, the server will exit with an error message.

 Insert Screenshot: Backend server running in terminal

3.2 Environment Configuration

The backend requires a .env file in the backend/ directory containing the following variables:

```
PORT=4000
MONGODB_URI=<your MongoDB Atlas connection string>
JWT_SECRET=<your secret key>
JWT_EXPIRES_IN=7d
GROQ_API_KEY=<optional Groq API key for AI review summaries>
```

4. Swagger UI Setup

Swagger UI is integrated into the backend using the `swagger-jsdoc` and `swagger-ui-express` packages. All API endpoints are annotated with OpenAPI 3.0 JSDoc comments in the route files, and the generated specification is served as an interactive documentation page.

Swagger UI URL: `http://localhost:4000/api-docs`

Once the backend is running, open a browser and navigate to the above URL. The Swagger UI page will display all 32 documented endpoints grouped into the following tags: Auth, Products, Users, Orders, Comments, and Flags.



Insert Screenshot: Swagger UI homepage showing all endpoint groups

4.1 Authorizing with a JWT Token

Protected endpoints require a valid JWT token. To authorize in Swagger UI:

- Log in via POST `/api/auth/login` to receive a token.
- Click the Authorize button (lock icon) at the top right of the Swagger page.
- In the Value field, paste the token directly (Swagger prepends Bearer automatically).
- Click Authorize, then Close.

All subsequent requests will include the `Authorization: Bearer <token>` header automatically.



Insert Screenshot: Swagger Authorize modal with token entered

5. Authentication Testing

Authentication is handled via JWT tokens. The `/api/auth` endpoints allow users to register and log in. Tokens expire after 7 days and must be included in the Authorization header for all protected routes.

5.1 Register Buyer

Endpoint: POST `/api/auth/register`

HTTP Method: POST

Role Required: Public (no token required)

Purpose: Register a new buyer account in the system.

Request Body:

```
{
  "name": "Buyer Test",
  "email": "buyer@test.com",
  "password": "Password123",
  "role": "buyer"
}
```

Expected Response: 201 Created — returns a JWT token and the new user object.

 [Insert Screenshot: Buyer registration request and 201 response](#)

Result: The server creates the buyer account in MongoDB and returns a token ready for use. The user object includes id, name, email, and role.

5.2 Login Buyer

Endpoint: POST `/api/auth/login`

HTTP Method: POST

Role Required: Public (no token required)

Purpose: Authenticate an existing buyer and receive a JWT token.

Request Body:

```
{
  "email": "buyer@test.com",
  "password": "Password123"
}
```

Expected Response: 200 OK — returns a JWT token and user profile data.

 [Insert Screenshot: Buyer login response with token](#)

Result: Login is successful. The returned token must be copied and pasted into the Swagger Authorize dialog to access protected buyer endpoints.

5.3 Register Seller

Endpoint: POST /api/auth/register

HTTP Method: POST

Role Required: Public (no token required)

Purpose: Register a new seller account with a business name.

Request Body:

```
{
  "name": "Seller Test",
  "email": "seller@test.com",
  "password": "Password123",
  "role": "seller",
  "businessName": "Test Store"
}
```

Expected Response: 201 Created — returns a JWT token and the new seller user object.

 [Insert Screenshot: Seller registration response](#)

Result: The seller account is created successfully. The businessName field is stored and returned in the user object.

5.4 Swagger Authorization

After obtaining a token from the login response, it must be submitted to Swagger's Authorize dialog to enable testing of protected routes. Click the Authorize lock icon at the top of the page, paste the token value, and click Authorize.

 [Insert Screenshot: Swagger Authorize modal with bearer token entered](#)

Once authorized, the lock icon on protected endpoints will appear closed, indicating that the token will be sent with each request.

6. Product API Testing

The product endpoints allow sellers to create and manage their product listings, and provide buyers with browsing and filtering capabilities. Seller-specific endpoints enforce the seller role via JWT.

6.1 Create Product

Endpoint: POST /api/products

HTTP Method: POST

Role Required: Seller (JWT required)

Purpose: Create a new product listing. The product is associated with the authenticated seller's account.

Request Body:

```
{
  "name": "Wireless Headphones",
  "description": "Noise-cancelling over-ear headphones",
  "price": 99.99,
  "category": "Electronics",
  "inventory": 50,
  "deliveryDays": 3
}
```

Expected Response: 201 Created — returns the newly created product object including `_id`, `sellerId`, and timestamps.

 [Insert Screenshot: Create product request and 201 response](#)

Result: The product is created and stored in MongoDB. The returned object includes the product ID which is needed for subsequent cart, order, and comment tests.

6.2 Get All Products

Endpoint: GET /api/products

HTTP Method: GET

Role Required: Public (no token required)

Purpose: Retrieve all active products. Supports optional query parameters: `q` (search by name) and `category` (filter by category).

Expected Response: 200 OK — returns an array of product objects, each enriched with seller information.

 [Insert Screenshot: GET /api/products response showing product list](#)

Result: All active products are returned successfully. Each product object includes seller name, business name, and contact details alongside the product data.

6.3 Get Product Details

Endpoint: GET /api/products/{id}

HTTP Method: GET

Role Required: Public (no token required)

Purpose: Retrieve a single product by its MongoDB ObjectId, including seller information and the 20 most recent reviews.

Expected Response: 200 OK — returns a product object and a comments array.

 [Insert Screenshot: GET /api/products/{id} response](#)

Result: The product detail endpoint returns all product fields, the associated seller profile, and recent comments in a single response.

6.4 Get Seller's Product List

Endpoint: GET /api/products/seller/me/list

HTTP Method: GET

Role Required: Seller (JWT required)

Purpose: Retrieve all products belonging to the authenticated seller, enriched with the total number of units sold from delivered orders.

Expected Response: 200 OK — returns an array of seller's products, each with an orders field showing total sold units.

 [Insert Screenshot: Seller product list response](#)

Result: The seller's product list is returned with sales data. The orders field aggregates sold quantities from delivered orders.

7. User API Testing

The Users API covers profile management for both buyers and sellers, as well as cart, wishlist, address management, and the cart checkout workflow. All endpoints require a valid JWT token for the appropriate role.

7.1 Get Seller Profile

Endpoint: GET /api/users/seller/me/profile

HTTP Method: GET

Role Required: Seller (JWT required)

Purpose: Retrieve the logged-in seller's profile information including business name, contact details, account balance, and average rating.

Expected Response: 200 OK — returns seller profile object.

 [Insert Screenshot: Get seller profile response](#)

Result: The seller profile is returned successfully with all business and contact fields.

7.2 Update Seller Profile

Endpoint: PATCH /api/users/seller/me/profile

HTTP Method: PATCH

Role Required: Seller (JWT required)

Purpose: Update selected fields of the seller's profile. Only the fields included in the request body are modified.

Request Body:

```
{
  "name": "Seller Test Updated",
  "businessName": "Updated Test Store",
  "email": "seller@test.com",
  "supportEmail": "support@teststore.com",
  "phone": "01000000000",
  "addressLine": "Test Street",
  "city": "Cairo",
  "country": "Egypt"
}
```

Expected Response: 200 OK — returns the updated seller profile object.

 [Insert Screenshot: Update seller profile response](#)

Result: Only the submitted fields are updated. Fields not included in the request body remain unchanged in the database.

7.3 Get Buyer Profile

Endpoint: GET /api/users/buyer/me/profile

HTTP Method: GET

Role Required: Buyer (JWT required)

Purpose: Retrieve the logged-in buyer's profile including name, email, phone, address, and saved delivery addresses.

Expected Response: 200 OK — returns buyer profile object with addresses array.

 [Insert Screenshot: Get buyer profile response](#)

Result: The buyer profile is returned including the full list of saved delivery addresses.

7.4 Update Buyer Profile

Endpoint: PATCH /api/users/buyer/me/profile

HTTP Method: PATCH

Role Required: Buyer (JWT required)

Purpose: Update selected personal fields of the buyer's profile.

Request Body:

```
{
  "name": "Buyer Test Updated",
  "email": "buyer@test.com",
  "phone": "01000000000",
  "addressLine": "Buyer Street",
  "city": "Cairo",
  "country": "Egypt"
}
```

Expected Response: 200 OK — returns the updated buyer profile object.

 [Insert Screenshot: Update buyer profile response](#)

Result: The buyer's profile fields are updated successfully. The email uniqueness constraint is enforced — a 409 is returned if the email is already in use by another account.

7.5 Add Buyer Delivery Address

Endpoint: POST /api/users/buyer/me/addresses

HTTP Method: POST

Role Required: Buyer (JWT required)

Purpose: Add a new delivery address to the buyer's saved addresses. The first address is automatically set as default.

Request Body:

```
{
  "label": "Home",
}
```

```
"line1": "123 Main Street",  
"city": "Cairo",  
"country": "Egypt",  
"postalCode": "12345",  
"isDefault": true  
}
```

Expected Response: 201 Created — returns the updated addresses array.

 [Insert Screenshot: Add buyer delivery address response](#)

Result: The address is added to the buyer's profile and at least one address is required before checkout can be performed.

7.6 Delete Buyer Delivery Address

Endpoint: DELETE /api/users/buyer/me/addresses/{addressId}

HTTP Method: DELETE

Role Required: Buyer (JWT required)

Purpose: Remove a saved delivery address. At least one address must remain — the last address cannot be deleted.

Expected Response: 200 OK — returns the updated addresses array after deletion.

 [Insert Screenshot: Delete buyer delivery address response](#)

Result: The address is removed. If the deleted address was the default, the first remaining address is automatically promoted to default.

7.7 Get Buyer Cart

Endpoint: GET /api/users/buyer/me/cart

HTTP Method: GET

Role Required: Buyer (JWT required)

Purpose: Retrieve the buyer's current cart with live product pricing, discounts, subtotal, and item count.

Expected Response: 200 OK — returns cart object with items array, subtotal, and itemCount.

 [Insert Screenshot: Get buyer cart response](#)

Result: The cart payload includes real-time pricing data fetched from the product collection. Inactive or deleted products are automatically excluded.

7.8 Add Product to Cart

Endpoint: POST /api/users/buyer/me/cart/items

HTTP Method: POST

Role Required: Buyer (JWT required)

Purpose: Add a product to the cart. If the product already exists in the cart, the quantity is incremented.

Request Body:

```
{
  "productId": "<productId>",
  "quantity": 2
}
```

Expected Response: 201 Created — returns the updated full cart payload.

 [Insert Screenshot: Add product to cart response](#)

Result: The product is added to the cart. If the same product is added again, the quantity increases rather than creating a duplicate entry.

7.9 Update Cart Item Quantity

Endpoint: PATCH /api/users/buyer/me/cart/items/{productId}

HTTP Method: PATCH

Role Required: Buyer (JWT required)

Purpose: Update the quantity of a specific product in the cart. Setting quantity to 0 removes the item.

Request Body:

```
{
  "quantity": 3
}
```

Expected Response: 200 OK — returns the updated cart payload.

 [Insert Screenshot: Update cart item quantity response](#)

Result: The quantity is updated. A quantity of 0 effectively removes the item — no separate delete call is needed for zero-quantity updates.

7.10 Remove Product from Cart

Endpoint: DELETE /api/users/buyer/me/cart/items/{productId}

HTTP Method: DELETE

Role Required: Buyer (JWT required)

Purpose: Remove a specific product from the buyer's cart.

Expected Response: 200 OK — returns the updated cart payload after removal.

 [Insert Screenshot: Remove product from cart response](#)

Result: The specific item is removed from the cart. All remaining items and the recalculated subtotal are returned.

7.11 Clear Buyer Cart

Endpoint: DELETE /api/users/buyer/me/cart

HTTP Method: DELETE

Role Required: Buyer (JWT required)

Purpose: Remove all items from the buyer's cart at once.

Expected Response: 200 OK — returns an empty cart object: { items: [], subtotal: 0, itemCount: 0 }.

 [Insert Screenshot: Clear buyer cart response](#)

Result: All cart items are removed in one operation. This is typically called after a successful checkout.

7.12 Checkout Cart

Endpoint: POST /api/users/buyer/me/cart/checkout

HTTP Method: POST

Role Required: Buyer (JWT required)

Purpose: Convert the buyer's entire cart into one or more orders. Items are grouped by seller — a separate order is created per seller. Stock is decremented atomically.

Request Body:

```
{
  "paymentMethod": "Cash on Delivery"
}
```

Expected Response: 201 Created — returns a checkout success message with the created orders array and order count.

 [Insert Screenshot: Checkout cart response with created orders](#)

Result: The checkout is successful. The cart is cleared and separate orders are created for each seller. Stock levels are reduced immediately.

7.13 Get Buyer Wishlist

Endpoint: GET /api/users/buyer/me/wishlist

HTTP Method: GET

Role Required: Buyer (JWT required)

Purpose: Retrieve all products saved in the buyer's wishlist. Only active products are returned.

Expected Response: 200 OK — returns an array of full product objects sorted by newest first.

 [Insert Screenshot: Get buyer wishlist response](#)

Result: The wishlist returns full product data for each saved item, allowing the buyer to review product details directly.

7.14 Add Product to Wishlist

Endpoint: POST /api/users/buyer/me/wishlist/{productId}

HTTP Method: POST

Role Required: Buyer (JWT required)

Purpose: Save a product to the buyer's wishlist. If the product is already on the wishlist, the operation is idempotent — no duplicate is created.

Expected Response: 201 Created — returns the updated wishlist IDs array.

 [Insert Screenshot: Add product to wishlist response](#)

Result: The product is added to the wishlist. Duplicate entries are prevented by the \$ne condition in the update query.

7.15 Remove Product from Wishlist

Endpoint: DELETE /api/users/buyer/me/wishlist/{productId}

HTTP Method: DELETE

Role Required: Buyer (JWT required)

Purpose: Remove a specific product from the buyer's wishlist.

Expected Response: 200 OK — returns the updated wishlist IDs array after removal.

 [Insert Screenshot: Remove product from wishlist response](#)

Result: The product is removed from the wishlist successfully.

8. Order Testing

Order management endpoints enable buyers to view their purchase history and cancel orders, while sellers can manage incoming orders and update their status through the fulfilment workflow.

8.1 Get Buyer Order History

Endpoint: GET /api/orders/buyer/me

HTTP Method: GET

Role Required: Buyer (JWT required)

Purpose: Retrieve a full list of the authenticated buyer's orders, enriched with product names, images, seller names, and any ratings given.

Expected Response: 200 OK — returns an array of detailed order objects sorted newest first.

 [Insert Screenshot: Buyer order history response](#)

Result: All buyer orders are returned with complete item details. The sellerRating field reflects any rating the buyer gave the seller for each order.

8.2 Get Seller Orders

Endpoint: GET /api/orders/seller/me

HTTP Method: GET

Role Required: Seller (JWT required)

Purpose: Retrieve all incoming orders for the authenticated seller, enriched with buyer information and product names.

Expected Response: 200 OK — returns an array of order objects assigned to this seller.

 [Insert Screenshot: Seller incoming orders response](#)

Result: The seller can view all orders with buyer delivery addresses, payment method, and current status.

8.3 Update Order Status

Endpoint: PATCH /api/orders/{id}/status

HTTP Method: PATCH

Role Required: Seller (JWT required)

Purpose: Advance an order through the fulfilment workflow. Allowed status values: Placed, Processing, Preparing, Shipping, Delivered, Cancelled. Seller earnings are credited automatically when status rules are met.

Request Body:

```
{  
  "status": "Processing"
```

```
}
```

Expected Response: 200 OK — returns the updated order object.

 [Insert Screenshot: Update order status response](#)

Result: The order status is updated. For Credit Card orders, the seller balance is credited when the order enters Processing. For Cash on Delivery, earnings are credited upon Delivered.

8.4 Cancel Order

Endpoint: PATCH /api/orders/buyer/{id}/cancel

HTTP Method: PATCH

Role Required: Buyer (JWT required)

Purpose: Cancel an order that is in Placed, Processing, or Preparing status. Stock is automatically rolled back upon cancellation.

Expected Response: 200 OK — returns the cancelled order object.

 [Insert Screenshot: Cancel order response](#)

Result: The order is cancelled and product inventory is restored. Orders in Shipping or Delivered status cannot be cancelled.

9. Comments and AI Review Summary Testing

The comments system allows authenticated users to leave product reviews with a star rating. An AI-powered summary endpoint uses the Groq API to generate a natural language summary of all reviews, with a keyword-based fallback if no API key is configured.

9.1 Add Product Comment

Endpoint: POST /api/comments

HTTP Method: POST

Role Required: Authenticated (buyer or seller)

Purpose: Submit a review and star rating for a product. If the user has already reviewed the product, the existing review is updated with the new content.

Request Body:

```
{
  "productId": "<productId>",
  "text": "Good product and fast delivery.",
  "rating": 5
}
```

Expected Response: 201 Created — returns the new comment object. 200 OK if updating an existing review.

 [Insert Screenshot: Add product comment response](#)

Result: The review is saved and the product's average rating and review count are recalculated in the database immediately.

9.2 Get Product Comments

Endpoint: GET /api/comments/product/{productId}

HTTP Method: GET

Role Required: Public (no token required)

Purpose: Retrieve all reviews for a specific product, sorted from newest to oldest.

Expected Response: 200 OK — returns an array of comment objects.

 [Insert Screenshot: Get product comments response](#)

Result: All reviews for the product are returned. Each comment includes the rating, text, user ID, and creation timestamp.

9.3 Generate AI Review Summary

Endpoint: GET /api/comments/product/{productId}/summarize

HTTP Method: GET

Role Required: Public (no token required)

Purpose: Generate a 2–3 sentence natural language summary of all product reviews. Uses the Groq Llama model if an API key is configured, otherwise falls back to a keyword-based analysis.

Expected Response: 200 OK — returns summary text, sample size, average rating, and aiGenerated flag.

 [Insert Screenshot: AI review summary response](#)

Result: The summary is generated successfully. The aiGenerated field indicates whether the Groq AI or the local fallback was used.

10. Flags and Reporting Testing

The flags system allows buyers to report sellers for late delivery and sellers to report buyers for non-receipt of packages. Each flag is linked to an order and increments the reported user's flag counter.

10.1 Create a Flag

Endpoint: POST /api/flags

HTTP Method: POST

Role Required: Authenticated (buyer or seller)

Purpose: Submit a report against another user for a specific order. Buyers may report 'Late Delivery'; sellers may report 'Package Not Received'. Duplicate flags for the same order are rejected.

Request Body:

```
{
  "reportedUserId": "<userId>",
  "reason": "Late Delivery",
  "orderId": "<orderId>",
  "details": "Order was 2 weeks late with no update."
}
```

Expected Response: 201 Created — returns the created flag object.

 [Insert Screenshot: Create flag response](#)

Result: The flag is created and the reported user's flag count is incremented. The system validates that the reason matches the caller's role.

10.2 Get Buyer Flags

Endpoint: GET /api/flags/buyer/my-flags

HTTP Method: GET

Role Required: Buyer (JWT required)

Purpose: Retrieve all flags submitted by or filed against the authenticated buyer.

Expected Response: 200 OK — returns an array of flag objects with populated user references.

 [Insert Screenshot: Buyer flags response](#)

Result: The buyer can view both flags they submitted and any flags raised against them, providing full transparency.

10.3 Get Seller Reports

Endpoint: GET /api/flags/seller/reports

HTTP Method: GET

Role Required: Seller (JWT required)

Purpose: Retrieve all flags involving the authenticated seller — both filed by the seller and filed against the seller.

Expected Response: 200 OK — returns up to 100 flag objects, newest first, with populated user fields.

 [Insert Screenshot: Seller reports response](#)

Result: The seller dashboard shows all relevant dispute reports, allowing the seller to review open issues.

10.4 Resolve a Flag

Endpoint: DELETE /api/flags/{id}

HTTP Method: DELETE

Role Required: Authenticated (original filer only)

Purpose: Resolve and remove a flag. Only the user who submitted the flag can delete it. The reported user's flag count is decremented.

Expected Response: 200 OK — returns a confirmation message.

 [Insert Screenshot: Resolve flag response](#)

Result: The flag is removed and the reported user's counter is decremented. Attempting to delete a flag filed by another user returns 403 Forbidden.

11. Error Handling Testing

Proper error handling is essential for API reliability. The Market_Place backend returns standardized error responses for all failure cases. The following table summarises the HTTP error codes used across the API:

Code	Status	Description and Example
400	Bad Request	Returned when required fields are missing, values are invalid, or business rules are violated. Example: placing an order with an empty items array, or providing a price below zero.
401	Unauthorized	Returned when no JWT token is provided or the token is expired/invalid. Example: calling POST /api/comments without an Authorization header.
403	Forbidden	Returned when the authenticated user does not have the required role. Example: a seller attempting to place an order (buyer-only endpoint).
404	Not Found	Returned when the requested resource does not exist. Example: fetching a product by an ID that does not exist in the database.
409	Conflict	Returned when a uniqueness constraint is violated. Example: registering with an email address that is already in use.
500	Internal Server Error	Returned for unexpected server-side errors. All errors are caught and returned as a JSON object with a message field.

 [Insert Screenshot: 401 Unauthorized response from a protected endpoint](#)

 [Insert Screenshot: 403 Forbidden response from a wrong-role request](#)

 [Insert Screenshot: 400 Bad Request with validation error message](#)

 [Insert Screenshot: 409 Conflict duplicate email registration error](#)

12. Conclusion

This report documented the Swagger UI-based API testing process for the Market_Place backend. Testing was performed across all 32 documented endpoints, covering the full range of system functionality including user authentication, product management, buyer and seller profile operations, shopping cart and wishlist workflows, order placement and fulfilment, product reviews with AI-powered summarization, and the user reporting system.

The test results confirmed that the backend API correctly implements the following behaviours:

- JWT-based authentication and role-based access control are enforced consistently across all protected endpoints.
- Buyer and seller registration and login flows return valid tokens and accurate user data.
- Product creation, retrieval, update, and deletion operate correctly with appropriate ownership enforcement.
- Cart operations — add, update, remove, and clear — reflect real-time product pricing and inventory data.
- The checkout endpoint successfully groups cart items by seller and creates separate orders, with atomic stock deduction.
- Order status transitions follow the defined workflow and seller earnings are credited according to payment method rules.
- The review system allows authenticated users to add or update product ratings, and the AI summary endpoint generates meaningful natural language summaries.
- The flags system correctly enforces role-based reporting rules and prevents duplicate submissions.
- All error scenarios return appropriate HTTP status codes and descriptive JSON messages.

Swagger UI proved to be an effective and efficient tool for API testing during development. Its integration directly into the backend server eliminates the need for external tooling, provides clear visibility into request and response schemas, and serves as living API documentation for the project.

The Market_Place API is fully functional and ready for integration with the buyer and seller frontend applications.